

Week 2: Exploratory Data Analysis (EDA) & Visualisation

Mentor: Doris Mugah

Date: 22 September 2026

Session type: Technical / Hands-On

Dataset: Brazilian E-Commerce Public Dataset by Olist

Session goal

Use simple visualisations to discover **patterns, relationships and outliers** in real data.

By the end of this beginner-friendly session, you should be able to:

- Explain EDA in simple terms.
- Inspect a dataset before plotting.
- Create charts with **Matplotlib** and **Seaborn**.
- Understand distributions.
- Compare categories.
- Explore relationships between numerical variables.
- Spot possible outliers.
- Create a simple correlation heatmap.
- Ask useful questions about a chart.
- Write a short interpretation of a visualisation.

Key idea: Do not create a chart just because you can. Start with a question.

1. What is EDA?

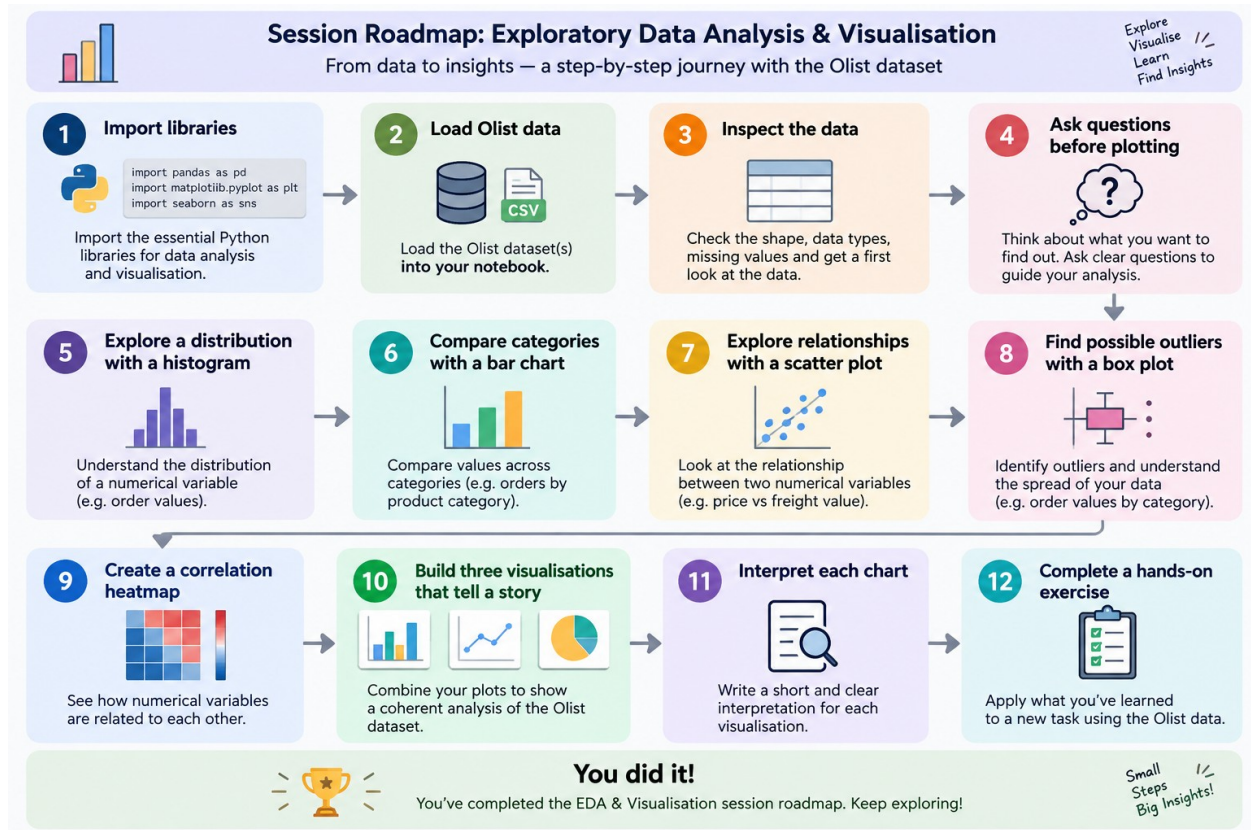
Exploratory Data Analysis (EDA) means exploring a dataset before doing advanced analysis or machine learning.

Think of it as **getting to know your data**.

We normally ask questions such as:

- What columns do I have?
- How many rows are there?
- Are values missing?
- What values are common?
- Are there unusual values?
- Are two variables related?
- What story does the data appear to tell?

2. Session roadmap



Import the libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os

sns.set_theme(style="whitegrid")#Adds a white background with
gridlines, making charts easier to read and compare.
pd.set_option("display.max_columns", None)#Displays all columns
without truncation.

print("Libraries loaded successfully!")

Libraries loaded successfully!
```

3. Load the Olist data

If you already downloaded the Olist CSV files, put them inside a folder called `olist_dataset`.

The most important tables for today's lesson are:

- olist_orders_dataset.csv
- olist_order_items_dataset.csv
- olist_products_dataset.csv
- olist_customers_dataset.csv

```
import os
from google.colab import userdata

# Install Kaggle API client
!pip install kaggle

# Set up Kaggle authentication using Colab secrets
os.environ['KAGGLE_USERNAME'] = userdata.get('KAGGLE_USERNAME')
os.environ['KAGGLE_KEY'] = userdata.get('KAGGLE_KEY')

# Define the dataset path on Kaggle
KAGGLE_DATASET_PATH = 'olistbr/brazilian-ecommerce'

# Create a directory for the dataset if it doesn't exist
DATA_DIR = './olist_dataset'
os.makedirs(DATA_DIR, exist_ok=True)

# Download the dataset
!kaggle datasets download -d {KAGGLE_DATASET_PATH} -p {DATA_DIR} --
unzip

print(f"Dataset downloaded and extracted to {DATA_DIR}/")

Requirement already satisfied: kaggle in
/usr/local/lib/python3.13/dist-packages (2.0.2)
Requirement already satisfied: bleach in
/usr/local/lib/python3.13/dist-packages (from kaggle) (6.4.0)
Requirement already satisfied: jupyter in
/usr/local/lib/python3.13/dist-packages (from kaggle) (1.19.5)
Requirement already satisfied: kagglesdk<1.0,>=0.1.20 in
/usr/local/lib/python3.13/dist-packages (from kaggle) (0.1.37)
Requirement already satisfied: packaging in
/usr/local/lib/python3.13/dist-packages (from kaggle) (26.3)
Requirement already satisfied: protobuf in
/usr/local/lib/python3.13/dist-packages (from kaggle) (5.29.6)
Requirement already satisfied: python-dateutil in
/usr/local/lib/python3.13/dist-packages (from kaggle) (2.9.0.post0)
Requirement already satisfied: python-slugify in
/usr/local/lib/python3.13/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: requests in
/usr/local/lib/python3.13/dist-packages (from kaggle) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dist-
packages (from kaggle) (4.67.3)
Requirement already satisfied: urllib3>=1.15.1 in
/usr/local/lib/python3.13/dist-packages (from kaggle) (2.5.0)
```

Requirement already satisfied: webencodings in
/usr/local/lib/python3.13/dist-packages (from bleach->kaggle) (0.6.1)

Requirement already satisfied: markdown-it-py>=1.0 in
/usr/local/lib/python3.13/dist-packages (from jupyter-text->kaggle)
(4.2.0)

Requirement already satisfied: mdit-py-plugins in
/usr/local/lib/python3.13/dist-packages (from jupyter-text->kaggle)
(0.6.1)

Requirement already satisfied: nbformat in
/usr/local/lib/python3.13/dist-packages (from jupyter-text->kaggle)
(5.11.1)

Requirement already satisfied: pyyaml in
/usr/local/lib/python3.13/dist-packages (from jupyter-text->kaggle)
(6.0.3)

Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.13/dist-packages (from python-dateutil->kaggle)
(1.17.0)

Requirement already satisfied: text-unidecode>=1.3 in
/usr/local/lib/python3.13/dist-packages (from python-slugify->kaggle)
(1.3)

Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.13/dist-packages (from requests->kaggle)
(3.4.9)

Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.13/dist-packages (from requests->kaggle) (3.19)

Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.13/dist-packages (from requests->kaggle)
(2026.7.22)

Requirement already satisfied: mdurl~0.1 in
/usr/local/lib/python3.13/dist-packages (from markdown-it-py>=1.0->jupyter-text->kaggle) (0.1.2)

Requirement already satisfied: fastjsonschema>=2.15 in
/usr/local/lib/python3.13/dist-packages (from nbformat->jupyter-text->kaggle) (2.22.2)

Requirement already satisfied: jsonschema>=2.6 in
/usr/local/lib/python3.13/dist-packages (from nbformat->jupyter-text->kaggle) (4.26.0)

Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in
/usr/local/lib/python3.13/dist-packages (from nbformat->jupyter-text->kaggle) (5.9.1)

Requirement already satisfied: traitlets>=5.1 in
/usr/local/lib/python3.13/dist-packages (from nbformat->jupyter-text->kaggle) (5.7.1)

Requirement already satisfied: attrs>=22.2.0 in
/usr/local/lib/python3.13/dist-packages (from jsonschema>=2.6->nbformat->jupyter-text->kaggle) (26.1.0)

Requirement already satisfied: jsonschema-specifications>=2023.03.6 in
/usr/local/lib/python3.13/dist-packages (from jsonschema>=2.6->nbformat->jupyter-text->kaggle) (2025.9.1)

```
Requirement already satisfied: referencing>=0.28.4 in
/usr/local/lib/python3.13/dist-packages (from jsonschema>=2.6-
>nbformat->jupyter-text->kaggle) (0.37.0)
Requirement already satisfied: rpds-py>=0.25.0 in
/usr/local/lib/python3.13/dist-packages (from jsonschema>=2.6-
>nbformat->jupyter-text->kaggle) (2026.6.3)
Requirement already satisfied: platformdirs>=2.5 in
/usr/local/lib/python3.13/dist-packages (from jupyter-core!
=5.0.*,>=4.12->nbformat->jupyter-text->kaggle) (4.11.7)
Warning: Looks like you're using an outdated `kaggle` version
(installed: 2.0.2), please consider upgrading to the latest version
(2.2.2)
Dataset URL: https://www.kaggle.com/datasets/olistbr/brazilian-
ecommerce
License(s): CC-BY-NC-SA-4.0
Downloading brazilian-ecommerce.zip to ./olist_dataset
100% 42.6M/42.6M [00:00<00:00, 184MB/s]
```

```
Dataset downloaded and extracted to ./olist_dataset/
```

```
# Load the main Olist tables into Pandas DataFrames
```

```
import os
```

```
orders = pd.read_csv(os.path.join(DATA_DIR,
"olist_orders_dataset.csv"))
order_items = pd.read_csv(os.path.join(DATA_DIR,
"olist_order_items_dataset.csv"))
products = pd.read_csv(os.path.join(DATA_DIR,
"olist_products_dataset.csv"))
customers = pd.read_csv(os.path.join(DATA_DIR,
"olist_customers_dataset.csv"))
sellers = pd.read_csv(os.path.join(DATA_DIR,
"olist_sellers_dataset.csv"))
order_payments = pd.read_csv(os.path.join(DATA_DIR,
"olist_order_payments_dataset.csv"))
order_reviews = pd.read_csv(os.path.join(DATA_DIR,
"olist_order_reviews_dataset.csv"))
```

```
print("Orders:", orders.shape)
print("Order items:", order_items.shape)
print("Products:", products.shape)
print("Customers:", customers.shape)
print("Sellers:", sellers.shape)
print("Payments:", order_payments.shape)
print("Reviews:", order_reviews.shape)
```

```
Orders: (99441, 8)
Order items: (112650, 7)
```

```
Products: (32951, 9)
Customers: (99441, 5)
Sellers: (3095, 4)
Payments: (103886, 5)
Reviews: (99224, 7)
```

4. First look at the data

Before making a graph, inspect the table.

`head()` shows the first five rows.

```
order_items.head()

{"type": "dataframe", "variable_name": "order_items"}
```

Check the size

`shape` returns:

(number of rows, number of columns)

```
print("Shape:", order_items.shape)
print("Rows:", order_items.shape[0])
print("Columns:", order_items.shape[1])
```

```
Shape: (112650, 7)
Rows: 112650
Columns: 7
```

Check the column names

```
order_items.columns.tolist()

['order_id',
 'order_item_id',
 'product_id',
 'seller_id',
 'shipping_limit_date',
 'price',
 'freight_value']
```

Check data types and missing values

`info()` gives us a quick summary of the DataFrame.

```
order_items.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 112650 entries, 0 to 112649
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   order_id              112650 non-null  object
1   order_item_id         112650 non-null  int64
2   product_id            112650 non-null  object
3   seller_id             112650 non-null  object
4   shipping_limit_date   112650 non-null  object
5   price                 112650 non-null  float64
6   freight_value         112650 non-null  float64
dtypes: float64(2), int64(1), object(4)
memory usage: 6.0+ MB

# Count missing values in each column
order_items.isnull().sum()

order_id              0
order_item_id         0
product_id            0
seller_id             0
shipping_limit_date   0
price                 0
freight_value         0
dtype: int64

```

Quick numerical summary

`describe()` shows useful statistics such as:

- **count** — number of non-missing values
- **mean** — average
- **std** — measures how spread out your data values are from the average
- **min** — smallest value
- **50%** — median
- **max** — largest value
- **25% (Q1)**— Approximately 25% of values are at or below
- **50% (Q2)**— Approximately 50% of values are at or below
- **75% (Q3)**— Approximately 75% of values are at or below

Note: standard deviation high means vary considerably, standard deviation is low close together

```
order_items[["price", "freight_value"]].describe()

{"summary": "{\n  \"name\": \"order_items[[\\\"price\\\", \\\"freight_value\\\"]]\",\n  \"rows\": 8,\n  \"fields\": [\n    {\n      \"column\": \"price\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 39528.033699224645,\n        \"min\": 0.85,\n        \"max\": 112650.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          120.65373901464716,\n          74.99,\n          112650.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"freight_value\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 39802.98023062691,\n        \"min\": 0.0,\n        \"max\": 112650.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          19.990319928983578,\n          16.26,\n          112650.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  },\n  \"type\": \"dataframe\"}
```

5. Ask a question before plotting

A good EDA workflow is:

Question → Choose variables → Choose chart → Plot → Interpret

Our first question is:

How are product prices distributed?

Because `price` is numerical, a **histogram** is useful.

Visualisation 1: Histogram — Distribution of product prices

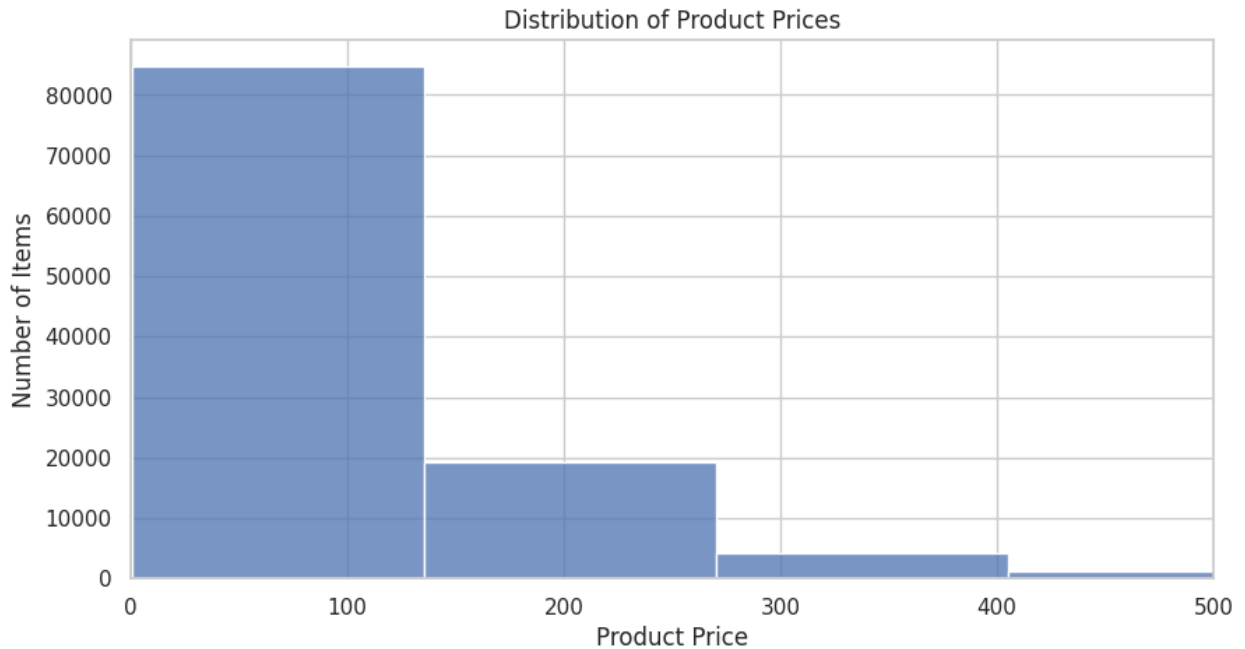
A histogram groups numerical values into ranges called **bins**.

It helps us see where most values are concentrated.

```
plt.figure(figsize=(10, 5))

sns.histplot(
    data=order_items,
    x="price",
    bins=50
)

plt.title("Distribution of Product Prices")
plt.xlabel("Product Price")
plt.ylabel("Number of Items")
plt.xlim(0, 500) # Focus on the majority of prices
plt.show()
```



How to interpret this chart

Look at the graph and ask:

1. Where are most product prices concentrated?
2. Is the distribution balanced or skewed?
3. Are there a few very expensive products?

Example interpretation:

Most item prices are concentrated in the lower price ranges. The distribution has a long right tail, which suggests that a smaller number of products are much more expensive than most products.

6. Categorical data

Categorical variables contain groups or labels.

Examples include:

- order status;
- customer state;
- product category.

Let's ask:

What are the most common order statuses?

```
# Count orders in each status
status_counts = orders["order_status"].value_counts()
```

```
status_counts
order_status
delivered      96478
shipped        1107
canceled        625
unavailable     609
invoiced        314
processing      301
created          5
approved         2
Name: count, dtype: int64
```

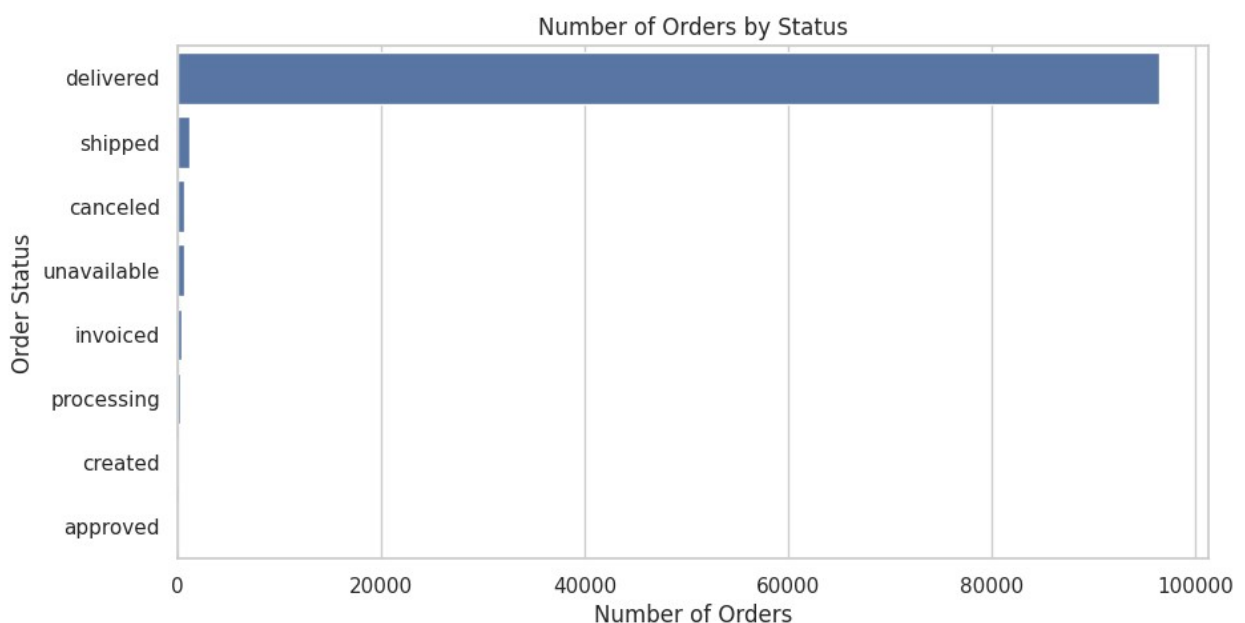
Visualisation 2: Bar chart — Orders by status

A bar chart is useful when comparing categories.

```
plt.figure(figsize=(10, 5))

sns.countplot(
    data=orders,
    y="order_status",
    order=orders["order_status"].value_counts().index
)

plt.title("Number of Orders by Status")
plt.xlabel("Number of Orders")
plt.ylabel("Order Status")
plt.show()
```



Interpretation questions

- Which status appears most often?
- Are some statuses rare?
- What might a business want to investigate further?

Example interpretation:

Delivered orders make up the largest group. Other statuses occur much less frequently, so the dataset is dominated by successfully delivered orders.

7. Relationships between variables

Sometimes we want to know whether two numerical variables appear related.

Our question is:

Do more expensive products also have higher freight costs?

A **scatter plot** is useful for comparing two numerical variables.

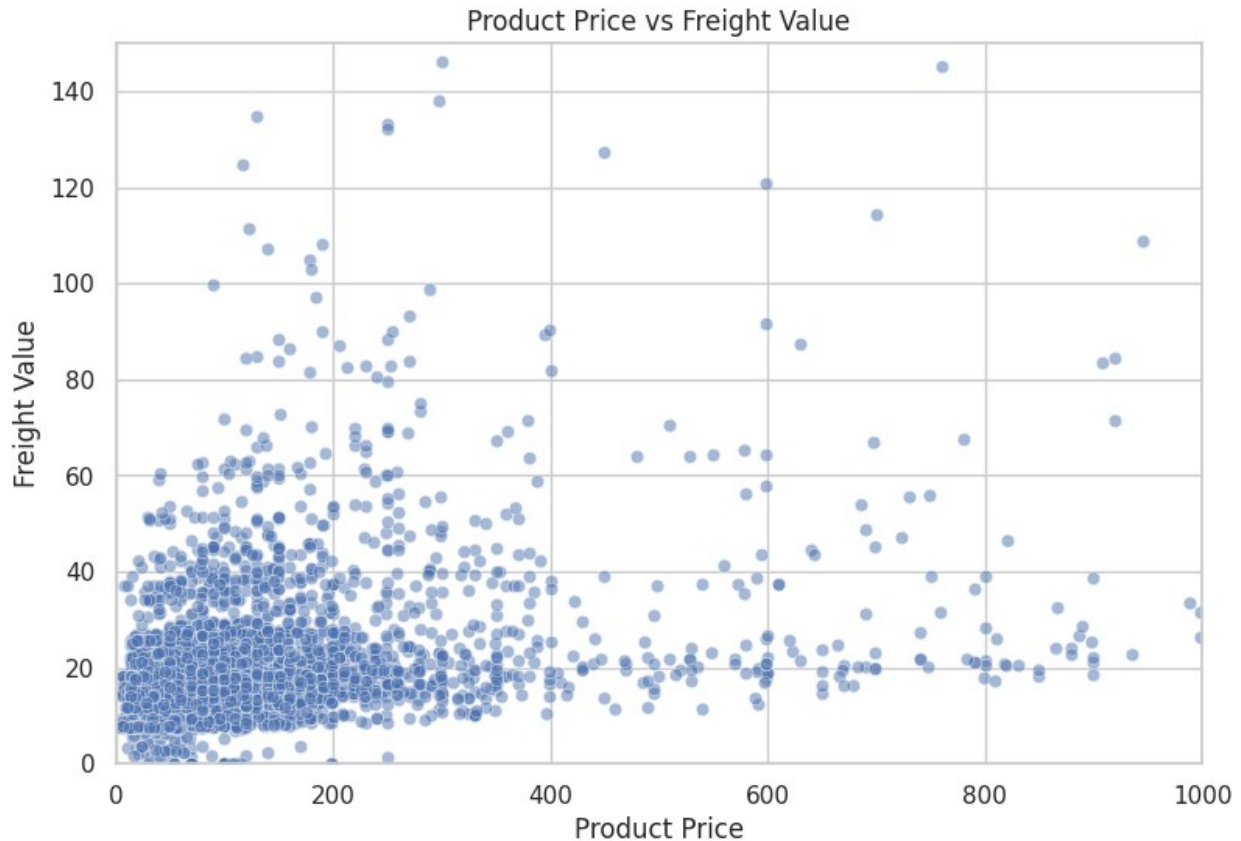
Visualisation 3: Scatter plot — Price vs freight value

```
# Take a sample so the graph remains easy to read
sample_items = order_items.sample(
    n=min(5000, len(order_items)),
    random_state=42
)

plt.figure(figsize=(9, 6))

sns.scatterplot(
    data=sample_items,
    x="price",
    y="freight_value",
    alpha=0.5
)

plt.title("Product Price vs Freight Value")
plt.xlabel("Product Price")
plt.ylabel("Freight Value")
plt.xlim(0, 1000)
plt.ylim(0, 150)
plt.show()
```



How to read a scatter plot

Each dot represents one observation.

Ask:

- Do the dots move upward together?
- Are they spread randomly?
- Are there unusual points far from the others?

Example interpretation:

There appears to be some relationship between item price and freight value, but the points are widely spread. Price alone therefore does not fully explain freight cost.

8. Outliers

An **outlier** is a value that is unusually far from most other observations.

Outliers are not automatically errors. They may represent:

- genuine expensive products;
- unusual orders;
- data-entry problems;

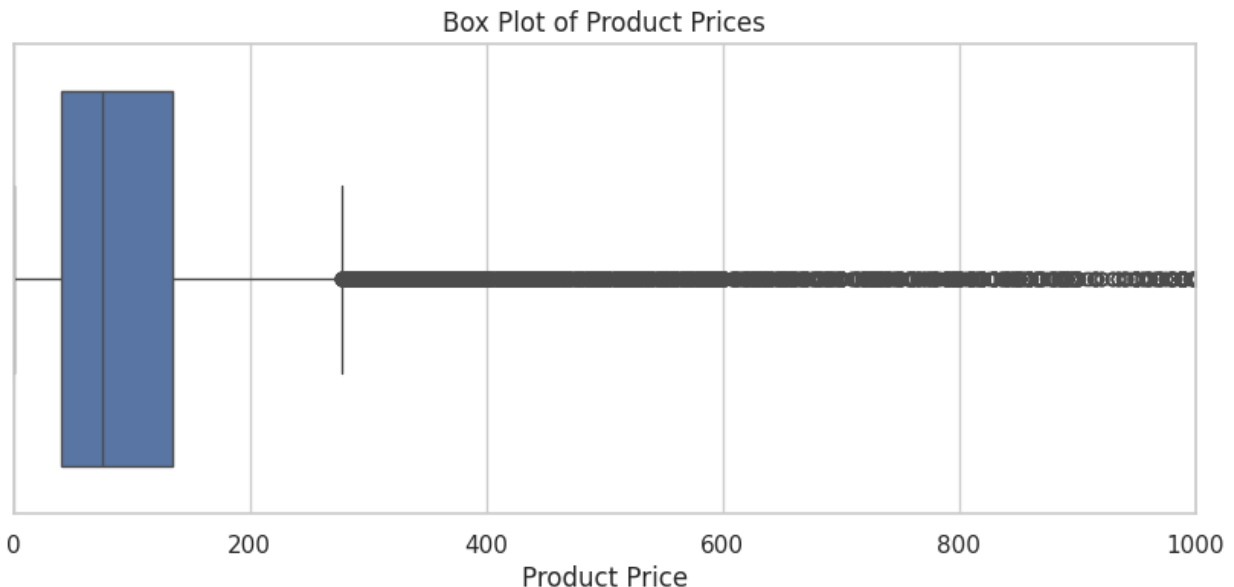
- rare but valid cases.

A **box plot** is a simple way to spot possible outliers.

```
plt.figure(figsize=(10, 4))

sns.boxplot(
    data=order_items,
    x="price"
)

plt.title("Box Plot of Product Prices")
plt.xlabel("Product Price")
plt.xlim(0, 1000)
plt.show()
```



Interpreting the box plot

The box shows the central portion of the data. Points far beyond the whiskers are possible outliers.

Important: Never delete an outlier simply because it looks unusual. Investigate it first.

```
# Look at the most expensive items
order_items.nlargest(10, "price")[
    ["order_id", "product_id", "price", "freight_value"]
]

{"summary": "{\n  \"name\": \"\",\n  \"rows\": 10,\n  \"fields\": [\n    {\n      \"column\": \"order_id\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 10,\n
```

```

\"samples\": [\n          \"b239ca7cd485940b31882363b52e6674\", \n
\"fefacc66af859508bf1a7934eable97f\", \n
\"8dbc85d1447242f3b127dda390d56e19\" \n          ], \n
\"semantic_type\": \"\", \n          \"description\": \"\" \n          } \n
    }, \n    { \n          \"column\": \"product_id\", \n
\"properties\": { \n          \"dtype\": \"string\", \n
\"num_unique_values\": 10, \n          \"samples\": [\n
\"dd1l3cb02b2af9c8e5787e8f1f0722f6\", \n
\"69c590f7ffc7bf8db97190b6cb6ed62e\", \n
\"259037a6a41845e455183f89c5035f18\" \n          ], \n
\"semantic_type\": \"\", \n          \"description\": \"\" \n          } \n
    }, \n    { \n          \"column\": \"price\", \n          \"properties\": { \n
          \"dtype\": \"number\", \n          \"std\": 1133.8739773811824, \n
          \"min\": 3999.9, \n          \"max\": 6735.0, \n
\"num_unique_values\": 10, \n          \"samples\": [\n
          4059.0, \n
          6729.0, \n
          4590.0 \n          ], \n
\"semantic_type\": \"\", \n          \"description\": \"\" \n          } \n
    }, \n    { \n          \"column\": \"freight_value\", \n
\"properties\": { \n          \"dtype\": \"number\", \n          \"std\": 65.86386765307836, \n
          \"min\": 17.01, \n          \"max\": 227.66, \n
          \"num_unique_values\": 10, \n          \"samples\": [\n
          104.51, \n
          193.21, \n
          91.78 \n          ], \n
\"semantic_type\": \"\", \n          \"description\": \"\" \n          } \n
    } \n  ] \n}, \"type\": \"dataframe\"}

```

9. Correlation heatmap

A **heatmap** can make relationships between numerical columns easier to see.

Correlation ranges from **-1 to +1**:

- close to **+1**: variables tend to increase together;
- close to **0**: weak/no linear relationship;
- close to **-1**: one tends to decrease as the other increases.

Correlation does **not** prove causation.

```

numeric_data = order_items[
    ["order_item_id", "price", "freight_value"]
]

correlation = numeric_data.corr()
correlation

{"summary": "{ \n  \"name\": \"correlation\", \n  \"rows\": 3, \n
\"fields\": [ \n    { \n          \"column\": \"order_item_id\", \n
\"properties\": { \n          \"dtype\": \"number\", \n          \"std\": 0.6035036054666065, \n
          \"min\": -0.0605220664366676, \n

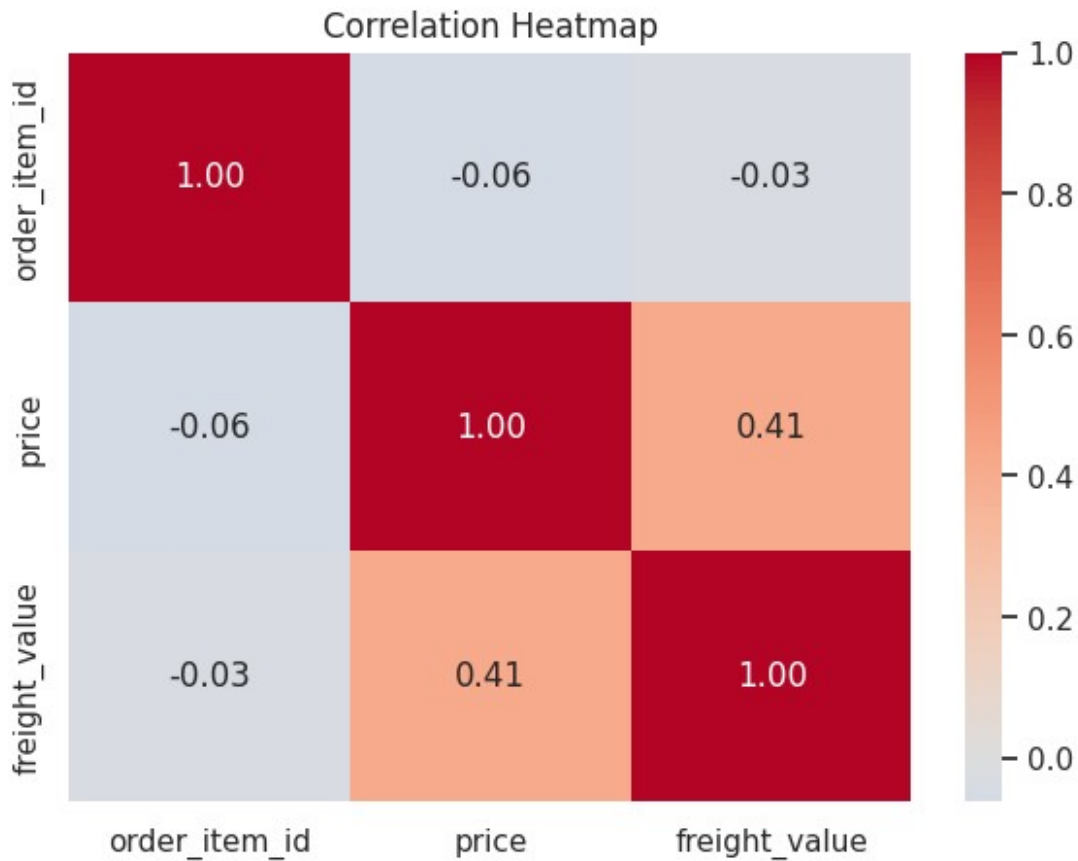
```

```
\\"max\\": 1.0,\n        \\"num_unique_values\\": 3,\n        \\"samples\\": [\n            1.0,\n            -0.0605220664366676,\n            0.02937977198122119\n        ],\n        \\"semantic_type\\": \\"\\",\n        \\"description\\": \\"\\",\n        \\"column\\": {\n            \\"price\\",\n            \\"properties\\": {\n                \\"dtype\\": \\"number\\",\n                \\"std\\": 0.5312295135227524,\n                \\"min\\": -0.0605220664366676,\n                \\"max\\": 1.0,\n                \\"num_unique_values\\": 3,\n                \\"samples\\": [\n                    -0.0605220664366676,\n                    1.0,\n                    0.41420431036303457\n                ],\n                \\"semantic_type\\": \\"\\",\n                \\"description\\": \\"\\",\n                \\"column\\": {\n                    \\"freight_value\\",\n                    \\"properties\\": {\n                        \\"dtype\\": \\"number\\",\n                        \\"std\\": 0.5163245334526458,\n                        \\"min\\": -0.02937977198122119,\n                        \\"max\\": 1.0,\n                        \\"num_unique_values\\": 3,\n                        \\"samples\\": [\n                            0.02937977198122119,\n                            0.41420431036303457,\n                            1.0\n                        ]\n                    },\n                    \\"description\\": \\"\\",\n                    \\"column\\": {\n                    }\n                }\n            }\n        }],\n        \\"type\\": \"dataframe\", \"variable_name\": \"correlation\"}
```

```
plt.figure(figsize=(7, 5))
```

```
sns.heatmap(
    correlation,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0
)
```

```
plt.title("Correlation Heatmap")
plt.show()
```



Heatmap discussion

Ask learners:

- Which pair has the strongest positive correlation?
- Is the relationship strong or weak?
- Does correlation tell us **why** two variables are related?

The numbers inside the heatmap are more important than the colour alone.

13. Quick recap

Today we learned:

```
df.head()
df.shape
df.info()
df.describe()
df.value_counts()

sns.histplot()
```

```
sns.countplot()  
sns.scatterplot()  
sns.boxplot()  
sns.heatmap()  
  
plt.title()  
plt.xlabel()  
plt.ylabel()  
plt.show()
```

Remember the EDA process

Ask → Explore → Visualise → Interpret → Ask another question

A good analyst does not only create graphs. A good analyst explains what the graphs mean.